The Yocto Project SDK



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Overview

- An SDK (Software Development Kit) is a set of tools allowing the development of applications for a given target (operating system, platform, environment, ...).
- It generally provides a set of tools including:
  - Compilers or cross-compilers.
  - Linkers.
  - Library headers.
  - Debuggers.
  - Custom utilities.



The Yocto Project SDK

- OpenEmbedded is used to generate images, by building many applications and doing a lot of configuration work.
  - When developing an application, we only care about the application itself.
  - We want to be able to develop, test and debug easily.
- The Yocto Project SDK is an application development SDK, which can be generated to provide a full environment compatible with the target.
- It includes a toolchain, libraries headers and all the needed tools.
- This SDK can be installed on any computer and is self-contained. The presence of OpenEmbedded is not required for the SDK to fully work.



Available SDKs

- Two different SDKs can be generated:
  - A generic SDK, including:
    - A toolchain.
    - Common tools.
    - A collection of basic libraries.
  - An image-based SDK, including:
    - The generic SDK.
    - The sysroot matching the target root filesystem.
- The toolchain in the SDKs is self-contained (linked to an SDK embedded libc).
- The SDKs generated with OpenEmbedded are distributed in the form of a shell script.
- Executing this script extracts the tools and sets up the environment.



The generic SDK

- Mainly used for low-level development, where only the toolchain is needed:
  - Bootloader development.
  - Kernel development.
- The recipe `meta-toolchain` generates this SDK:
  - `bitbake meta-toolchain`
- The generated script, containing all the tools for this SDK, is in:
  - `$BUILDDIR/tmp/deploy/sdk`
  - Example: `poky-glibc-x86_64-meta-toolchain-cortexa8hf-neon-toolchain-6.0.sh`
- The SDK will be configured to be compatible with the specified MACHINE.



The image-based SDK

- Used to develop applications running on the target.
- One task is dedicated to the process. The task behavior can vary between the images.
  - `populate_sdk`
- To generate an SDK for `core-image-minimal`:
  - `bitbake -c populate_sdk core-image-minimal`
- The generated script, containing all the tools for this SDK, is in:
  - `$BUILDDIR/tmp/deploy/sdk`
  - Example: `poky-glibc-x86_64-core-image-minimal-cortexa8hf-neon-toolchain-6.0.sh`
- The SDK will be configured to be compatible with the specified MACHINE.



Adding packages to the SDK

- Two variables control what will be installed in the SDK:
  - TOOLCHAIN_TARGET_TASK: List of target packages to be included in the SDK
  - TOOLCHAIN_HOST_TASK: List of host packages to be included in the SDK
- Both can be appended to install more tools or libraries useful for development.
- Example: to have native `curl` on the SDK:

```
TOOLCHAIN_HOST_TASK:append = " nativesdk-curl"
```



SDK format

- Both SDKs are distributed as shell scripts.
- These scripts self extract themselves to install the toolchains and the files they provide.
- To install an SDK, retrieve the generated script and execute it.
  - The script asks where to install the SDK. Defaults to `/opt/poky/<version>`
  - Example: `/opt/poky/6.0`

```
$ ./poky-glibc-x86_64-meta-toolchain-cortexa8hf-neon-toolchain-6.0.sh
Poky (Yocto Project Reference Distro) SDK installer version 6.0
==========================================================
Enter target directory for SDK (default: /opt/poky/6.0):
You are about to install the SDK to "/opt/poky/6.0". Proceed[Y/n]?
Extracting SDK.................done
Setting it up...done
SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source
the environment setup script e.g.
 $ . /opt/poky/6.0/environment-setup-cortexa8hf-neon-poky-linux-gnueabi
```



Use the SDK

- To use the SDK, a script is available to set up the environment:

```
$ cd /opt/poky/6.0
$ source ./environment-setup-cortexa8hf-neon-poky-linux-gnueabi
```

- The `PATH` is updated to take into account the binaries installed alongside the SDK.
- Environment variables are exported to help using the tools.



SDK installation

environment-setup-cortexa8hf-neon-poky-linux-gnueabi
: Exports environment variables.

site-config-cortexa8hf-neon-poky-linux-gnueabi
: Variables used during the toolchain creation

sysroots
: SDK binaries, headers and libraries. Contains one directory for the host and one for the target.

version-cortexa8hf-neon-poky-linux-gnueabi
: Version information.



SDK environment variables

- CC: Full path to the C compiler binary.
- CFLAGS: C flags, used by the C compiler.
- CXX: C++ compiler.
- CXXFLAGS: C++ flags, used by `CPP`
- LD: Linker.
- LDFLAGS: Link flags, used by the linker.
- ARCH: For kernel compilation.
- CROSS_COMPILE: For kernel compilation.
- GDB: SDK GNU Debugger.
- OBJDUMP: SDK objdump.

To see the full list, open the environment script.



Examples

- To build an application for the target:

```
$ $CC -o example example.c
```

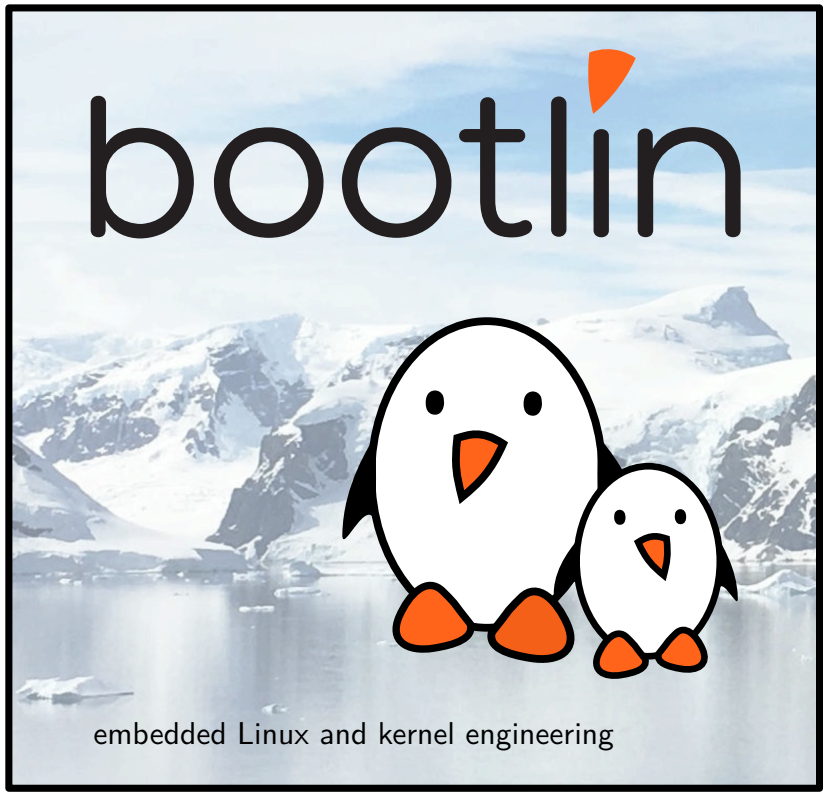
- The LDFLAGS variable is set to be used with the C compiler (`gcc`).

Practical lab - Create and use a Yocto SDK



- Generate an SDK
- Compile an application for the target in the SDK

Devtool



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Overview

- `Devtool` is a set of utilities to ease the integration and the development of OpenEmbedded recipes.
- It can be used to:
  - Generate a recipe for a given upstream application.
  - Modify an existing recipe and its associated sources.
  - Upgrade an existing recipe to use a newer upstream application.
- `Devtool` adds a new layer, automatically managed, in `$BUILDDIR/workspace/`.
- It then adds or appends recipes to this layer so that the recipes point to a local path for their sources. In `$BUILDDIR/workspace/sources/`.
  - Local sources are managed by `git`.
  - All modifications made locally should be committed.



devtool usage 1/3

There are three ways of creating a new `devtool` project:

- To create a new recipe: `devtool add <recipe> <fetchuri>`
  - Where `recipe` is the recipe's name.
  - `fetchuri` can be a local path or a remote *uri*.
- To modify the source for an existing recipe: `devtool modify <recipe>`
- To upgrade a given recipe: `devtool upgrade -V <version> <recipe>`
  - Where `version` is the new version of the upstream application.



devtool usage 2/3

Once a `devtool` project is started, commands can be issued:

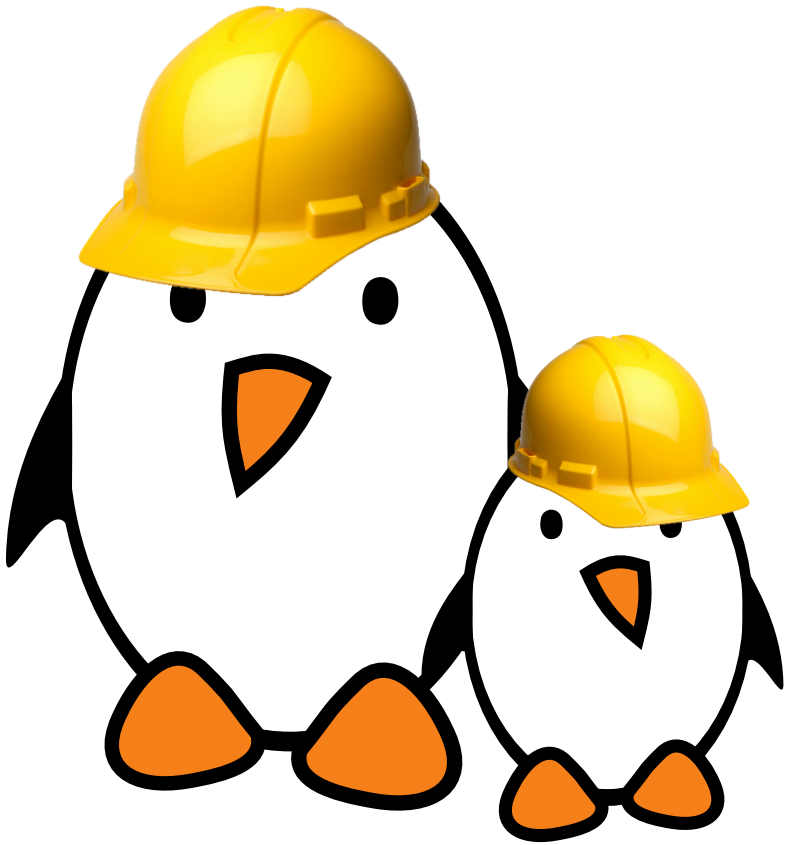
- `devtool edit-recipe <recipe>`: edit `recipe` in a text editor (as defined by the `EDITOR` environment variable).
- `devtool build <recipe>`: build the given `recipe`.
- `devtool build-image <image>`: build `image` with the additional `devtool` recipes' packages.



devtool usage 3/3

- `devtool deploy-target <recipe> <target>`: upload the `recipe`'s packages on `target`, which is a live running target with an SSH server running (`user@address`).
- `devtool update-recipe <recipe>`: generate patches from git commits made locally.
- `devtool reset <recipe>`: remove `recipe` from the control of `devtool`. Standard layers and remote sources are used again as usual.

Practical lab - Using devtool



- Generate a new recipe
- Modify a recipe to add a new patch
- Upgrade a recipe to a newer version